

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В.Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)

Институт информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Направление подготовки 09.03.04 Программная инженерия

Направленность (профиль) образовательной программы Разработка программно-информационных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

**Разработка сервиса хранения аудиоданных с применением алгоритмов
упаковки и потоковой распаковки**

Студент: Пахомов Владислав Андреевич

Зав. кафедрой: канд. техн. наук, доц. Поляков Владимир Михайлович

Руководитель: канд. физ.-мат. наук, доц. Осипов Олег Васильевич

К защите допустить

Зав. кафедрой _____ /Поляков В.М./

« _____ » _____ 2026 г.

Белгород 2026 г.

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В.Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)

Институт информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Направление подготовки 09.03.04 Программная инженерия

Направленность (профиль) образовательной программы Разработка программно-информационных систем

Утверждаю:

Зав. кафедрой _____

« _____ » _____ 2026 г.

ЗАДАНИЕ

на выпускную квалификационную работу студента

Пахомова Владислава Андреевича

1. Вид выпускной квалификационной работы (ВКР): *бакалаврская работа.*
2. Тема ВКР *Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения* утверждена приказом по университету от «28» января 2026 г. №2/124.
3. Срок сдачи студентом законченной ВКР: «12» июня 2026 г.
4. Исходные данные: ключевые слова, название алгоритмов и методов, технологии разработки и т.п. Пример: рекомендательные системы, подходы построения рекомендательных систем, методы матричной факторизации, оптимизационные методы обучения модели, аффинные преобразования, трёхмерная графика, машинное обучение, клиент-серверное приложение, генеративная нейронная сеть.
5. Содержание ВКР: Здесь должны быть перечислены названия разделов (глав) пояснительной записки. Пример:
 - Введение,
 - Описание предметной области прогнозирования погоды,
 - Разработка архитектуры программного обеспечения прогнозирования погоды,
 - Разработка алгоритмов и программного обеспечения прогнозирования погоды,
 - Тестирование программной системы прогнозирования погоды,
 - Руководство пользователя разработанной системы прогнозирования погоды,
 - Заключение,
 - Список литературы,
 - Приложение А,
 - Приложение Б.

6. Перечень графического материала: 1278 рисунков, 1785 таблиц. Презентация включает: постановку задачи, актуальность, технологии, численные эксперименты, внешний вид программы, заключение.

Консультанты по работе с указанием относящихся к ним разделов:

Раздел	Консультант	Задание выдал (подпись, дата)	Задание принял (подпись, дата)

Дата выдачи задания: «17» января 2026 г.

(подпись руководителя)

Задание принял (приняла) к исполнению

(подпись студента)

КАЛЕНДАРНЫЙ ПЛАН

№ п/п	Наименование этапов работы	Срок выполнения этапов работы	Примечание
1	Анализ предметной области. Постановка задачи разработки системы прогнозирования погоды методами машинного обучения.	18.01.26 – 15.02.26	Выполнено
2	Разработка алгоритмов, структур нейросетей и архитектуры системы прогнозирования погоды.	15.02.26 – 20.03.26	Выполнено
3	Разработка программной части системы прогнозирования погоды.	20.03.26 – 29.03.26	Выполнено
4	Разработка клиентской части системы прогнозирования погоды.	29.03.26 – 29.04.26	Выполнено
5	Тестирование программного обеспечения для прогнозирования погоды.	29.04.26 – 17.05.26	Выполнено
6	Оформление пояснительной записки. Подготовка выступления.	17.05.26 – 10.06.26	Выполнено
5	Тестирование программного обеспечения для прогнозирования погоды.	29.04.26 – 17.05.26	Выполнено
6	Оформление пояснительной записки. Подготовка выступления.	17.05.26 – 10.06.26	Выполнено

Студент (ка) _____ *Пахомов Владислав Андреевич*
(подпись) (Ф.И.О.)

Руководитель ВКР _____ *Осипов Олег Васильевич*
(подпись) (Ф.И.О.)

Результаты проверки ЭВ ВКР на заимствование

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Студент (ка)	Пахомов В.А.	ПВ-223	16.06.2026
	(Ф.И.О.)	(подпись)	(дата)

Тема ВКР: Разработка сервиса хранения аудиоданных с применением алгоритмов упаковки и потоковой распаковки.

ВКР прошла проверку на объём заимствований.

Итоговая оценка заимствований: **0%**.

Работу проверил

канд. физ.-мат. наук, доцент		16.06.2026	Хлопов А.М.
(уч. степень, должность)	(подпись)	(дата)	(Ф.И.О.)

Руководитель ВКР

канд. социол. наук, доцент		16.06.2026	Островский А.М.
(уч. степень, должность)	(подпись)	(дата)	(Ф.И.О.)

ОПРЕДЕЛЕНИЯ, СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ

РФ — Российская Федерация.

СВО — специальная военная операция.

ВУЗ — высшее учебное заведение.

ВКР — выпускная квалификационная работа.

ИТ — информационные технологии.

БД — база данных.

ПО — программное обеспечение.

СУБД — система управления базами данных.

ООП — объектно-ориентированное программирование.

ЭВМ — электронно-вычислительная машина.

КОП — код операции.

АЛУ — арифметико-логическое устройство.

ПЛИС — программируемая логическая интегральная схема.

БПФ — быстрое преобразование Фурье.

СЛАУ — система линейных алгебраических уравнений.

ОС — операционная система.

API (Application Programming Interface) — программный интерфейс, обеспечивающий взаимодействие между компонентами программного обеспечения.

RGB (red, green, blue) — цветовой формат машинной графики, основанный на смешении компонентов красного, зелёного и синего цветов.

WWW (World Wide Web) — всемирная паутина, система взаимосвязанных гипертекстовых документов.

PDF (Portable Document Format) — переносимый формат электронных документов.

HTML (HyperText Markup Language) — язык гипертекстовой разметки, который используется для создания и структурирования веб-страниц.

SQL (Structured Query Language) — язык структурированных запросов, используемый для работы с базами данных.

VPN (Virtual Private Network) — виртуальная частная сеть, обеспечивающая защищённое соединение через интернет.

ОПРЕДЕЛЕНИЯ, СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ							
Изм.	Лист	№ докум.	Подп.	Дата			
Разраб.	Пахомов В.А.		3.06.26	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Руковод.	Осипов О.В.		3.06.26			4	27
Консул.					БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.		11.06.26				
Зав. каф.	Поляков В.М.		25.06.26				

IoT (Internet of Things) — интернет вещей, концепция подключения различных устройств к интернету для обмена данными.

JSON (JavaScript Object Notation) — текстовый формат обмена данными, который используется для хранения и передачи структурированной информации между системами и приложениями.

					ОПРЕДЕЛЕНИЯ, СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		5

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	10
1.1 Актуальность разрабатываемого сервиса	10
1.2 Анализ существующих решений	11
1.2.1 Анализ сервисов хранения аудиоданных	11
1.2.2 Анализ алгоритмов хранения данных	14
1.3 Процесс уменьшения размерности информации	17
1.3.1 Автоэнкодер	17
1.3.2 Обзор архитектуры перцептронов	18
1.3.3 Обзор преобразования входных данных	19
1.3.4 Функции потерь	20
1.4 Постановка задачи	21
1.5 Средства решения задач проекта	21
1.5.1 Язык программирования Python	21
1.5.2 СУБД Postgres	23
1.5.3 Nginx	23
1.5.4 Javascript	24
1.5.5 Инструмент для контейнеризации Docker	26

					<i>СОДЕРЖАНИЕ</i>			
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подп.</i>	<i>Дата</i>				
<i>Разраб.</i>		<i>Пахомов В.А.</i>		<i>3.06.26</i>	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	<i>Лит.</i>	<i>Лист</i>	<i>Листов</i>
<i>Руковод.</i>		<i>Осипов О.В.</i>		<i>3.06.26</i>			6	27
<i>Консул.</i>						БГТУ им. В.Г. Шухова, ПБ-223		
<i>Н. контр.</i>		<i>Бондаренко Т.В.</i>		<i>11.06.26</i>				
<i>Зав. каф.</i>		<i>Поляков В.М.</i>		<i>25.06.26</i>				

ВВЕДЕНИЕ

Мир современного человека всё больше переходит в цифровое пространство, и это не удивительно. Цифровое пространство обладает неоспоримыми преимуществами при работе с информацией:

- Дешёвое хранение информации, а значит её большой объём
- Большая скорость передачи информации
- Возможность обрабатывать большое количество информации очень быстро
- Качество хранимой информации

Эти факторы так или иначе и стали решающими в решении хранения данных как в быту человека, так и в бизнес-процессах и в творчестве.

Самым ярким и приближённым к цифровому пространству примером является профессия программиста. На это влияет как сама специфика и предназначение профессии, так и вышеописанные процессы. Можно ли себе представить программиста, который совсем не владеет компьютером? Конечно же нет. Возьмут ли на работу программиста, который продолжает хранить весь свой код на листах бумаги? Только если эта компания почему-то любит запускать очень старые ракеты. Современного конкурентноспособного разработчика сложно представить без умных текстовых редакторов, хранения кода в облаке, автоматизации процессов доставки клиентам, нейронных сетей помогающих в написании кода (а иногда ещё и пишущих за них).

Сама профессия по личному мнению автора лежит на стыке творческого процесса и бизнес процессов в разной степени важности в зависимости от разрабатываемого продукта. Она требует как изобретательности, креативности и нестандартных подходов при решении различных задач, так и соответствия конкретным рамкам, условиям и отлаженным процессам.

На примере профессии программиста можно увидеть, насколько сильно может интегрироваться цифровое пространство в сферу требующую как творческого подхода, так и точности, и насколько много пользы оно может принести при решении задач. Важно, однако, отметить, что цифровое пространство всё ещё не может полностью вытеснить другие подходы. Это такой же инструмент, как, например, молоток. Оно решает конкретные проблемы, связанные преимущественно с информацией.

Одна из сфер, которая очень похожа на программирование, является музыкальная сфера. Она также требует творческого подхода, но также задаёт в себе

					ВВЕДЕНИЕ			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.	Пахомов В.А.			3.06.26				
Руковод.	Осипов О.В.			3.06.26			7	27
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.			11.06.26				
Зав. каф.	Поляков В.М.			25.06.26				

определённые рамки.

И да, за последнее время интеграция цифрового пространства в этой сфере тоже сильно продвинулась. Сервисы музыкального стриминга, согласно исследованиям, используют уже более трети россиян, а рост аудитории продолжается. В процессе написания музыки всё чаще используются DAW, умеющие работать с продвинутым оборудованием записи; синтез звука и его обработка тоже проходили очень долгий путь эволюции.

Однако на рынке продюсирования музыки почему-то очень мало инструментов, которые помогают в разработке аудиоданных. Множество продюсеров всё ещё складывают свои проекты в крайне неорганизованное хранилище часто представляющее собой обычную папку на локальных персональных компьютерах. А если данные будут утеряны, то обычно с этим ничего поделать нельзя и их приходится восстанавливать "по памяти".

И такие проблемы очень легко решить. Тем более, для них уже есть решение. И их можно позаимствовать из той же ранее рассмотренной профессии программиста. Облачное хранилище данных позволит избежать проблемы утери данных а также позволит делиться ими касанием пальца, что тоже повысит эффективность общения с коллегами. Хранение предыдущих версий позволит возвращаться к старым концепциям, пересматривать их, рефлексировать и адаптировать в разработке. Распространение данных в централизованной платформе гораздо более удобно для конечного потребителя в лице дистрибьютора или слушателя. Дополнение платформы в виде бизнес-сущностей позволит дистрибьюторам при публикации полученных композиций.

Создание единой платформы также создаёт множество задач по обработке и выдаче данных. Аудиоданные могут быть очень большими, особенно если это демонстрационные записи, не отличающиеся особой лаконичностью. В связи с этим должна быть также возможность порционной выдачи данных в реальном времени.

Первая глава посвящена анализу предметной области, в ней рассматривается актуальность создания сервиса, выполняется сравнительный анализ готовых решений, формируются функциональные и нефункциональные требования к разрабатываемому сервису.

Вторая глава описывает проектирование: общие архитектурные паттерны, рассмотрена клиент-серверная составляющая и интерфейсы их взаимодействия, архитектура хранилищ данных

Третья глава содержит структуру программного обеспечения: модели и серви-

					ВВЕДЕНИЕ	Лист
						8
Изм.	Лист	№ докум.	Подп.	Дата		

сы, конкретные библиотеки при реализации.

Четвёртая глава описывает инфраструктуру сервиса: процессы контейнеризации, выдачи статических данных, конфигурации Nginx и обратного проксирования, подключения графических устройств для работы алгоритма сжатия.

Разрабатываемый сервис соответствует концепциям и тенденциям развития отрасли, поддерживаемый и расширяемый. Цель реализуемого сервиса - управление процессами разработки композиции а также упрощение её распространения в профессиональной среде.

					ВВЕДЕНИЕ	Лист
						9
Изм.	Лист	№ докум.	Подп.	Дата		

1 ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Актуальность разрабатываемого сервиса

Разработка музыкальных композиций занимает одно из важных мест на сегодняшнем рынке. Количество потребителей музыкального контента каждый день продолжает расти, о чём говорит статистика музыкальных стриминговых сервисов. Однако инструментов для управления разработки и обеспечения её эффективности у современных продюсеров представлено очень мало.

На текущий момент самым распространённым способом хранения аудиоданных является их хранение на локальных носителях или же на хранителе персональных компьютеров. Повреждение или утеря хранилищ приведёт к утере данных а также к необходимости их восстановления с нуля, что будет занимать очень много времени а также имеет риск привести к невозможности восстановления изначально-го варианта ввиду несовершенства человеческой памяти.

Важным аспектом является развитие композиции. Один проект или же композиция могут иметь несколько различных форм. Например, может измениться инструментальная или вокальная партия. Кроме того, отслеживание предыдущих версий позволит найти более удачный вариант исполнения и использовать наработки в новой итерации. Также вид композиции может сильно разниться в зависимости от его целей и вариантов исполнений. В отношении живой музыки это может быть композиция записанная в живом исполнении или в студии. Путём введения запрета на редактирование и удаление данных можно обеспечить как улучшенные гарантии хранения данных, так и сохранение всех версий композиции.

Для возможности отделения музыкальных композиций по их назначению можно ввести понятие тегов. Тег - короткое строковое описание версии, которое описывает цель этого релиза.

При условии наличия большого количества версий композиции возникает вопрос составления серий композиций, или же плейлистов. Так как каждый из треков имеет свою определённую ценность и форму, плейлист так же может иметь свою ценность. Например, владельцу площадки для выступлений будет не очень интересно услышать, как группа будет звучать в живой записи, гораздо большую ценность для него будет предоставлять живое выступление. Плейлисты позволяют быть гораздо более гибким и предоставлять различным заказчикам интересную для

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.		Пахомов В.А.		3.06.26				
Руковод.		Осипов О.В.		3.06.26			10	27
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.		Бондаренко Т.В.		11.06.26				
Зав. каф.		Поляков В.М.		25.06.26				

его задач информацию.

Предоставление служебной информации также сильно облегчит процесс взаимодействия с проектом. В любой момент можно посмотреть название проекта, исполнителей, описание без дополнительного обращения, что позволит увеличить эффективность разработки.

Важным аспектом является налаживание коммуникации между командой разработчиков. Самым простым способом может быть система комментирования, позволяющая оставить наработки, мысли и отзывы команды разработки касательно композиции.

Возможность запустить сервис на собственных серверах может стать решающей для большинства компаний для обеспечения в первую очередь конфиденциальности информации.

Введение запрета на редактирование и удаление композиций неизбежно ведёт к использованию большого объёма памяти. Необходимо выбрать алгоритм, кодек или же разработать собственный подход для хранения аудиоданных.

Таким образом, разрабатываемый сервис позволит получить инструмент, обеспечивающий централизованное защищённое хранилище наработок в музыкальной сфере, позволяющее наладить рабочие процессы внутри команды а также наладить общение с заказчиками.

1.2 Анализ существующих решений

1.2.1 Анализ сервисов хранения аудиоданных

Готовых продуктов, обеспечивающих описанную логику, очень немного, вот несколько самых популярных из них:

- Git
- Splice
- DropBox
- Boombox.io

Проведём сравнительный анализ.

Git, хорошо известный в среде программирования, не является программным обеспечением, направленным именно на хранение аудиоданных или любых других больших медиа данных. Он лучше всего справляется именно с текстовой информацией. Кроме того, Git - это система контроля версий, а не отдельный сервис. Он хранит историю об изменениях, но не имеет привязки к какому-либо сервису. Это же и преимущество, так как декомпозиция на сервис и программу позволяет

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		11

публиковать свои данные на любой совместимый продукт, вроде GitHub, GitLab, BitBucket или же развёрнутый самостоятельно сервис. Систему плейлистов можно попробовать воссоздать внутри папки, а значит по возможности распространения он сильно уступает. Git обеспечивает надёжное хранение данных, использует концепцию невозможности удаления и изменения данных, очень надёжен и имеет систему тегов. Сервисы позволяют наладить отличную работу в команде с использованием запросов на слияние, комментариев. Git и сервисы невероятно гибки, но они дают слишком много свободы и предлагают слишком мало удобств для данной задачи.

Splice на сегодняшний день отошёл от концепции хранения версионных аудиоданных и стал скорее сервисом, содержащим сэмплы - короткие музыкальные фразы. Ранее сервис предлагал облачное хранилище для проектов в DAW и коллаборативную работу над ними в облачном пространстве. Всё ещё нет системы плейлистов и обозначения композиций и нет возможности развернуть собственное хранилище.

DropBox - этот вариант чуть лучше, чем хранение на диске. Как минимум, данные уже содержатся в облаке, а значит к ним есть доступ. Существуют инструменты для интеграции с DAW. Но это нельзя назвать полноценным сервисом для хранения аудиоданных. Это просто сервис хранения данных в облаке, который через плагины используется для хранения аудио. DropBox поддерживает создание тегов и версионирования файлов, создания своих свойств для файлов. Но создания обновляемых плейлистов всё ещё нет. Эта система не заточена именно под хранение аудиоданных.

Boombox.io - самый близко подобранный к требованиям сервис. Он умеет создавать версии треков, приватные плейлисты. Композиции снабжаются метаданными а также в нём много инструментов ИИ. Существует комментирование с указанием временных меток. Однако данные изменяемые - это значит, что хранимые они могут быть нарушены. Сервис находится исключительно в облаке и не может быть предоставлен кампаниям в виде сервиса для собственного развёртывания.

Таблица 1.1 – Сравнительная таблица сервисов хранения аудиоданных

	Git	Splice	DropBox	Boombox.io
Хранение данных в облаке	Зависит от сервиса	Есть, облачный сервис	Есть, облачный сервис	Есть, облачный сервис

Продолжение таблицы 1.1

	Git	Splice	DropBox	Boombox.io
Версионирование и тегирование	Есть, полная система	В данный момент устарела, отсутствует	Есть 180-дневная история изменений по платной подписке	Присутствуют версии, нет тегов
Инструменты коллаборации	Зависит от сервиса (комментарии, запросы на слияние и пр.); Отдельные ветки и слияние веток	Совместная работа в DAW	Совместная работа над файлами, комментирование	Комментарии с указанием таймкодов, обычные комментарии
Поддержка разворачивания собственного сервиса	Есть	Нет, облачный сервис	Нет, облачный сервис	Нет, облачный сервис
Создание плейлистов с указанием версий	Возможно, но неудобно	Нет	Возможно, но неудобно	Есть
Указание метаданных	Выполняется вручную - неудобно	Есть	Есть, нестандартизировано	Есть, выполняется полуавтоматически
Неизменность данных	Есть	Нет	Нет	Нет

Продолжение таблицы 1.1

	Git	Splice	DropBox	Boombox.io
Оптимизации для хранения аудиоданных	Выполняются вручную - неудобно	Не имеет значения	Выполняется вручную	Нет

Проведённое исследование показывает, что сервисы можно разделить на две категории.

Первая категория - это сервисы предназначенные для работы исключительно с одной конкретной задачей. Git решает вопрос версионирования, а DropBox решает проблему хранения в облаке. Обычно работа с такими инструментами требует дополнительной обработки, оптимизации и введения правил, по которым они будут храниться. Они предоставляют решение одной конкретной задачи и потому слишком универсальны, они не предоставляют оптимизаций для работы конкретно с аудио данными.

Вторая категория - сервисы, направленные непосредственно на работу с аудио. Начнём с того, что они крайне непопулярны. Их очень мало, и поэтому, чтобы продолжать выживать, они инкапсулируют свои собственные решения на своих серверах и предлагают их за плату. Такие решения могут быть угрозой конфиденциальности для бизнеса.

Моё решение будет брать лучшее из двух категорий. В нём можно найти специфичный для аудиоданных и их распространения функционал, а также решённые задачи версионирования и хранения данных в облаке. Кроме того, текущее решение распространяется в свободном доступе.

1.2.2 Анализ алгоритмов хранения данных

Алгоритмы кодирования аудиоданных прошли очень долгий путь развития, рассмотрим несколько из них

- MP3
- Opus
- FLAC
- WAV

MP3 - аудиокодек с потерей данных. Этот кодек использует психоакустические оптимизации для того, чтобы сократить количество занимаемого места. Из-за этих

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		14

оптимизаций, особенно на маленьком битрейте, может возникать очень множество артефактов при прослушивании. Звук становится наиболее чистым и неотличимым от исходного при 320 КБит/с. Однако, даже при большом битрейте, кодек всё ещё допускает потери данных, что недопустимо в профессиональной сфере.

Opus - современный кодек, используемый в основном для стриминга ввиду его очень маленького битрейта (около 96 КБит/с) при котором он становится неотличим от оригинального звука. Недостатком является то, что всё ещё очень мало устройств могут поддерживать этот формат, а также он даёт большую нагрузку для устройства воспроизводящего аудио. Кроме того, кодек тоже допускает потерю данных, что может быть заметно на хорошем оборудовании. Кодек Opus оптимизирован под одну конкретную частоту дискретизации - 48000 Гц, что так же ограничивает его использование в студиях.

FLAC - старый формат без потери данных. Он использует модифицированный алгоритм кодирования Хаффмана - кодирование сдвига Хаффмана. В фрейм записываются не повторяющиеся данные, а их линейный рост. Такой метод сжатия позволяет достичь коэффициента сжатия в 2 раза, что меньше чем предыдущие кодеки, однако и потери данных в нём нет.

WAV - кодек, содержащий необработанный уровень амплитуды без сжатия. Практически не влияет на загруженность устройства, однако и содержит в себе больше всего данных. Идеально подходит для записи и использования в DAW ввиду простоты его распаковки, но плох для длительного хранения.

Таблица 1.2 – Сравнительная таблица кодеков

	MP3	Opus	FLAC	WAV
Коэффициент сжатия	10	17	1.7	Без сжатия
Нагрузка при кодировании/декодировании	Средняя	Высокая	Низкая	Крайне низкая

Продолжение таблицы 1.2

	MP3	Opus	FLAC	WAV
Поддержка кодеков на устройствах	Очень хорошая поддержка на старых и новых устройствах	Работает на- тивно толь- ко на отно- сительно но- вых устрой- ствах	Хорошая поддержка на старых и новых устройствах	Может пло- хо работать на опера- ционных системах кроме Windows, в остальном довольно хорошая поддержка
В восстано- вленных дан- ных нет по- терь	Нет	Нет	Да	Да
Исполь- зование в стриминге	Хорошо под- ходит для стриминга	Лучше всего под- ходит для стриминга	Редко ис- пользуется в стриминге, только там где важно качество	Требуется очень боль- шого про- пускного канала для стриминга
Битрейт	320 КБит/с	96 КБит/с	800 КБит/с	1411 КБит/с
Влияние на исходный материал	Сильное ис- пользование психоаку- стических принципов	Сильное ис- пользование психоаку- стических принципов	Нет	Нет

Кодеки MP3 и Opus подходят для хранения аудиоданных для конечного потре-
бителя, так как включают в себя психоакустические приёмы: затухание, удаление
неслышимых данных, маскирование. При дополнительной обработке данных в DAW
утраченные данные могут повлиять на результат выполняемой работы, поэтому для
текущей задачи эти кодеки использовать можно, но только в определённых сценари-

ях, вроде подготовки выпуска композиции в финальных сервисах дистрибуции.

Кодеки FLAC и WAV же не искажают исходные данные, а значит они лучше подходят для композиций записанных в данном формате. Но их узким местом является количество занимаемых данных. FLAC частично решает эту проблему используя математические методы кодирования вместо психоакустических. Однако степень сжатия данных в особенно сложных местах может быть достаточно маленькой.

Можно модифицировать алгоритм FLAC и предоставить возможность кодировать особо сложные фреймы при помощи методов, допускающих потерю данных, однако не влияющих на исходную волну амплитуды достаточно сильно.

1.3 Процесс уменьшения размерности информации

Процессы уменьшения размерности информации это задачи которые позволяют сократить количество свойств внутри какого-либо объекта за счёт нахождения связи между свойствами этого объекта.

Существует множество способов, позволяющих сократить размерность данных. Математические способы обычно используют матрицу корреляции для определения взаимосвязи между свойствами внутри объекта. По этой же матрице данные можно восстановить обратно.

Однако аудиоданные часто могут содержать более комплексные связи, которые невозможно выразить при помощи таких матриц. В небольшом отрезке из аудиофайла могут быть как сильные всплески амплитуды, так и постоянные данные. Однако музыкальная композиция всё ещё подчиняется гармоническим частотным правилам, соотношениям частот внутри аккордов, темпу и ритму.

Для нахождения неочевидных и сложных закономерностей всё чаще применяются свёрточные нейросети (CNN).

1.3.1 Автоэнкодер

Автоэнкодер по своей структуре очень похож на обычную свёрточную нейросеть. Он так же состоит из перцептронов, у него есть скрытые слои, входной и выходной слои. Однако есть в архитектуре и различия.

Автоэнкодер можно разделить на две составляющие: энкодер и декодер. Задача энкодера - уменьшить размерность данных. Каждый из последующих слоёв уменьшает размерность данных. Получившийся вектор называют латентным вектором или же латентом. Он содержит в себе только самую важную информацию. Входным вектором для декодера как раз будет являться латентный вектор. Задача декодера - восстановить пробелы в латенте чтобы получить исходный объект. Если энкодер

шёл на уменьшение количества свойств объекта, то декодер уже увеличивает их количество в обратном порядке.

Чем меньше разрыв между количеством перцептронов в каждом слое, тем более качественным должен получиться результат упаковки/распаковки. Однако с увеличением количества перцептронов нейросеть будет обучаться гораздо дольше, проход будет выполняться дольше а ресурсов обеспечивающих обучение и использование нейросети может не хватить.

ВСТАВИТЬ КАРТИНКУ

Автоэнкодеры находят своё применение не только в задачах уменьшения размерности вектора, но и в задачах преобразования исходных данных. Например, удаление шума с изображения или любых других данных, выделение каких-либо признаков.

ВСТАВИТЬ КАРТИНКУ

У данной архитектуры нейронной сети для текущей задачи есть как положительные, так и отрицательные стороны. Часто для восстановления музыкального отрезка очень важен контекст: что звучало до текущего отрезка, так как контекст содержит информацию о волнах. Некоторые особенно низкие частоты могут быть и не определены как частота в виду достаточно маленького времени. Минимально слышимая для человеческого уха частота в 20 Гц потребует около 0,05 секунд для проигрывания одного периода. Проблему контекста в данной архитектуре можно решить при помощи увеличения фрейма, например, до 1 секунды. Этой информации будет достаточно для сохранения частот. Однако с увеличением входного слоя увеличивается сложность обучения, требования к аппаратному обеспечению растут.

Есть и плюсы в использовании текущего подхода. Декодирование отдельных независимых друг от друга фреймов позволит масштабировать процесс декодирования при появлении более продвинутого аппаратного обеспечения путём параллельного декодирования. Кроме того, автоэнкодер довольно просто обучать и использовать.

1.3.2 Обзор архитектуры перцептронов

Как уже было сказано ранее, автоэнкодер может быть достаточно тяжёлой свёрточной нейросетью при использовании большого количества входных параметров. Существуют способы оптимизации, позволяющие сократить количество связей между нейронами.

Полносвязная нейросеть

Полносвязный слой строится по принципу "нейрон связан с каждым нейроном

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		18

из предыдущего слоя”. В контексте аудиоданных это может быть как положительным примером, так и отрицательным. Так, например, более низкие частоты с большим периодом смогут влиять на значение текущего нейрона, а значит информация о низких частотах будет сохранена. Но полносвязные слои крайне неоптимизированны. Вместе с важными данными в нейрон может приходить и ”мусор который будет только добавлять шума в декодированной записи.

При использовании полносвязной нейросети требуется тонкая настройка входного слоя. Далеко не все данные могут быть полезными, и слишком большой входной вектор может привести к очень большому времени достижения экстремума для нейронной сети при обучении.

Использование Convolution слоёв

Convolutional слои позволяют создавать неполносвязную нейронную сеть. На примере одномерного массива Conv-слой сможет выбирать лишь небольшой участок из более большого вектора.

ВСТАВИТЬ КАРТИНКУ

Использование Convolutional слоёв позволяет решить проблему слишком большой выборки данных, что позволит уменьшить количество связей между нейронами. Уменьшение связей между нейронами позволит увеличить размер входного слоя, что позволит увеличить количество кодируемой/декодируемой информации за один раз. Однако при использовании такого рода слоёв всё ещё остаётся проблема настройки выборки. Кроме того, при потоковой распаковке аудио гораздо более ценна работа с относительно небольшим фреймом данных.

1.3.3 Обзор преобразования входных данных

Нейронные сети умеют работать с данными различных размерностей. Рассмотрим, в какие форматы данных можно было бы преобразовать аудиоданные.

Одномерный вектор

Это самый простой способ. Аудиоданные преобразуются в одномерный массив, свойство каждого из которых содержит амплитуду. Этот способ относительно лёгок в реализации и не требует больших преобразовательных мощностей, так как фактически выполняется преобразование аудиоданных в последовательность амплитуд, что естественно для большинства аудиоформатов на сегодняшний день, и производится дополнительная нормализация данных для работы в нейронной сети.

Двухмерная спектрограмма

Можно преобразовать аудиоданные в более сложный и требующий больших вычислений формат. Спектрограмма представляет собой изображение. Одна из

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		19

координат может представлять собой время, а другая - частоту. Яркость точки на изображении в точке (x, y) будет показывать, насколько сильна амплитуда частоты x в момент y . Для получения такого изображения используются алгоритм быстрого преобразования Фурье. Для этого способа так же требуется достаточно большой размер фрейма. Если фрейм будет недостаточно большим, то алгоритм просто "не услышит" длинные волны.

ВСТАВИТЬ КАРТИНКУ

Использование такого подхода так же будет вызывать трудности и с декодированием спектрограммы. Для этого тоже используются специальные алгоритмы Гриффина-Лима или обратного быстрого преобразования Фурье. Данный способ даёт большую нагрузку на аппаратное обеспечение и может вызывать задержки при потоковой распаковке.

1.3.4 Функции потерь

Функции потерь будут прямо влиять на процесс обучения и, соответственно, на получившееся звучание аудиоданных. Именно функция потерь определяет то, на что будет сфокусировано внимание при обучении.

MSELoss

Самая простая функция. Предположим, что у нас есть исходный сигнал y и предсказанный сигнал $\bar{y}$ размерность N . Тогда функция потерь MSE будет высчитана как

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \bar{y}_i)^2$$

Эта функция может быть не очень хороша для аудиоданных, так как она слабо учитывает отклонения во всплесках - в аудио это может быть удар бочки или же по тарелке. Но она неплохо восстанавливает общую картинку и общую слышимость.

Spectral Loss

Функция спектральной потери считает потери по определённым частотам. Эта функция потерь гораздо более близка к природе аудиоданных, так как работает с её частотами напрямую. Обычно высчитывается потеря по нескольким частотам отражающим одну ноту, например [128 Гц, 256 Гц, 512 Гц, 1024 Гц]. Для определения амплитуд частот используется быстрое преобразование Фурье. Эта функция потерь гораздо лучше сохраняет детали, нежели общее звучание. Семейство функций спектральной потери часто используется при обучении нейронных сетей, работающих с аудиоданными.

Положим, что у нас есть исходный сигнал y и $\bar{y}$. Тогда функция спектральной потери будет высчитана следующим образом

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		20

$$SpectralLoss = |\log(|STFT(y)| + \epsilon) - \log(|STFT(\bar{y})| + \epsilon)|$$

Где ϵ - очень маленькая величина.

Чаще всего используется комбинированная функция потери в зависимости от цели при обучении.

1.4 Постановка задачи

В соответствии с целью ВКР поставим список задач выполнение которых позволит достигнуть поставленной цели.

1. Проектирование бизнес-сущностей для обеспечения аудиоданных необходимыми служебными данными.
2. Разработка или поиск методов оптимизированного хранения аудиоданных.
3. Разработка архитектуры и программного обеспечения для хранения аудиоданных.
4. Тестирование разработанного программного обеспечения для хранения аудиоданных.
5. Создание руководства пользователя разработанного программного обеспечения для хранения аудиоданных.

1.5 Средства решения задач проекта

1.5.1 Язык программирования Python

Python - интерпретируемый язык программирования, разработанный и представленный в 1991 году. Python изначально планировался как язык программирования, очень похожий на язык псевдокода. Именно поэтому в нём не получится найти фигурные скобки, точки с запятыми. Исходя из этого подхода Python унаследовал не только лаконичность, но и также строгие синтаксические правила оформления кода (PEP8), без которых он не запустится.

Ввиду своей простоты язык программирования стал очень популярен в различных сферах как прототипирования, так и написания сложных серьёзных программ. Сейчас язык используется для описания прямых высокоуровневых команд и действий, будь это эндпоинт веб-сервера или же обращение к видеокарте. Это стало возможно благодаря тому, что большинство библиотек Python являются интерфейсами к исполняемым файлам, содержащим основную логику и ценность библиотеки.

Такой подход позволяет управлять сложной большой машиной через относительно лёгкий и понятный человеку интерфейс. Для поставленных задач язык программирования предоставляет специализированные инструменты.

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		21

Для работы с нейросетями предоставляется пакет Torch - это библиотека позволяющая создавать и обучать нейронные сети, умеющая работать с аппаратным обеспечением через интерфейсы CUDA, ROCm, Metal. Для работы с математическими вычислениями, например, при вычислении функции потерь можно использовать популярную библиотеку NumPy, предоставляющую широкий спектр математических функций и преобразований.

В качестве библиотеки, позволяющей развернуть свой собственный REST API веб-сервер, можно взять FastAPI. Эта библиотека предоставляет возможность быстрого создания точек обращения к интерфейсу взаимодействия с сервером с использованием системы Dependency Injection - внедрения зависимостей, позволяющей подключать к обслуживаемой точке только необходимые компоненты, будь то сервис или репозиторий.

Для контроля входных параметров а также обеспечения их валидности для Python предоставляется библиотека Pydantic, действующая по принципу Parse Don't Validate - паттерну, согласно которому каждый ввод обязательно должен быть преобразован в более высокоуровневые структуры данных обеспечивающих корректность данных. Чёткие правила валидации для каждого поля в API позволяет избежать возможных проблем с безопасностью в виде инъекций. Библиотеки семейства Pydantic также предлагают удобный инструментарий для работы с переменными среды и настройками проекта, установке фильтров и сортировок для выборки.

Для взаимодействий с базой данных предоставляется библиотека SQLAlchemy в сочетании с Alembic. Первая библиотека - это ORM, которая позволяет конструировать запросы к базе данных в зависимости от выбранного адаптера. Таким образом, сервер может в любой момент начать использовать вместо MySQL PostgreSQL. Alembic поддерживает актуальность текущей базы данных при помощи системы миграций: он автоматически сканирует модели проекта и на основе их содержания собирает набор инструкций, позволяющих актуализировать базу данных до указанных моделей.

Изобилие и разнообразие библиотек невозможно контролировать без пакетного менеджера, который будет следить за совместимостью всех указанных библиотек и версии Python, и для этого можно использовать пакетный менеджер Poetry. Он создаёт собственное окружение в зависимости от указанных библиотек.

Python - гибкий и универсальный язык программирования, с помощью которого можно написать любое программное обеспечение, и позволяющий сконцентрироваться в первую очередь на самых важных решениях.

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		22

1.5.2 СУБД Postgres

PostgreSQL - это современная объектно-реляционная СУБД с открытым исходным кодом. Она является альтернативой коммерческим решениям вроде Oracle Database, а также предоставляет широкий набор функционала, недоступный в других базах данных. На сегодняшний день PostgreSQL, согласно опросам разработчиков, является самым популярным выбором среди баз данных.

Postgres покрывает большинство потребностей, которые могут возникнуть у разработчиков благодаря своему набору функций. База данных поддерживает широкий список возможных индексаций для тонкой настройки, возможность использования JSON в качестве значений колонок а также поиск по этим значениям, высокая эффективность и утилизация подключений благодаря Multiversion concurrency control.

И даже если текущего функционала в Postgres недостаточно, всегда можно расширить его, сделав инструмент максимально подходящим для решения поставленных задач.

Самое главное - инструмент достаточно взрослый и для него существует очень много адаптеров и библиотек для работы с ним. Есть возможность настроить миграции и без проблем использовать Postgres в качестве базы данных для ORM благодаря высокой поддержке языка запросов SQL.

1.5.3 Nginx

Nginx - современный открытый веб-сервер, пришедший на смену коммерческому решению в виде Apache Server.

Главным преимуществом Nginx является то, что он умеет способен работать в многопоточном режиме. Обычная настройка Nginx уже довольно неплохо справляется с множественным обращением пользователей, но её можно изменить увеличив количество исполнителей и количество соединений, которое может обрабатывать исполнитель. Количество открытых соединений связано с ограничениями операционной системы и ограничено количеством открытых файлов, а количество исполнителей нет смысла ставить больше чем количества ядер процессора.

Ещё одной особенностью Nginx является продвинутый роутинг запросов при помощи настройки location в конфигурационном файле. При помощи продвинутого фильтра можно настроить выдачу файлов как с бекенда, так и отдавать данные необходимые для отображения фронтенда.

Nginx может выступать и как Прoxy сервер, перенаправляя запросы на внутренне открытые ресурсы если они доступны и наоборот отклонять запросы если

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		23

они ведут на защищённый ресурс.

Настройка сервера остаётся как достаточно лёгкой, так и достаточно гибкой что позволяет решать различные задачи связанные с перенаправлением на нужные ресурсы.

1.5.4 Javascript

JavaScript - это скриптовый язык программирования предназначенный для запуска внутри браузеров. Язык программирования строится на основе стандарта ECMAScript, разрабатываемый организацией ECMA. На основе языка программирования ECMAScript существуют приложения написанные для Node.JS, например веб-серверы, использующие Express.

JavaScript имеет специфичные для браузера API, вроде объекта window, позволяющий получить доступ к DOM страницы, navigator содержащий информацию о браузере. Кроме того, JavaScript в браузерах всегда однопоточный. Да, в JS есть генераторы и асинхронные функции, но они не являются по-настоящему отдельным потоком. Операции чтения и вызовы yield, await приостанавливают поток выполнения внутри функции и движок переключается на другую функцию.

Одной из особенностей JavaScript является то, что у него динамическая типизация. И это может быть как удобно, так и неудобно. Динамическая типизация позволяет написать код очень быстро, но поддерживать его становится очень сложно из-за того что контракты между модулями неявные.

Эту проблему решает расширение JavaScript - TypeScript, вводящий статическую типизацию и проверку типов на уровне компиляции. TypeScript, разработанный Microsoft и выпущенный в 2012 году, является высокоуровневым языком программирования со статической типизацией, построенный на основе JavaScript. TypeScript предоставляет возможность описывать свои интерфейсы, типы и классы а также на этапе компиляции проверяет их согласованность. Введение статической типизации позволяет писать более сложные программы.

Одним из наиболее популярных подходов на сегодняшний день при организации графического интерфейса программы является реактивное программирование. Это крайне удобный подход, позволяющий подписываться на изменения состояния и перерисовывать компоненты в зависимости от их изменения. Такой библиотекой для JavaScript является React.

В основе React лежит движок Fiber, которая отслеживает изменения состояния и перевыполняет функции отрисовки и просчётов внутри компонентов, лежащих в дереве. React после своего выхода начал быстро набирать свою популярность благо-

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
						24
Изм.	Лист	№ докум.	Подп.	Дата		

даря такому подходу. React решал несколько проблем: теперь зависимости внутри компонентов строго иерархические и переходили снизу-наверх, что увеличивало поддерживаемость кода; иерархичность а также автоматически отслеживаемое состояние позволяло быть компонентам всегда в самом актуальном состоянии; очень большая скорость отрисовки и перерисовки компонентов. Конечно, библиотеку можно сломать ненужными вызовами, обилием `useEffect` и применением прочих антипаттернов. Однако, это всё ещё довольно сложно, именно поэтому на просторах интернета всё ещё встречаются плохо написанные приложения React которые однако несмотря на плохую оптимизацию, продолжают работать достаточно быстро.

Существуют и другие библиотеки, использующие реактивный подход в других целях. Библиотека React может использоваться для отрисовки компонентов, однако бизнес-логику стоит делать независимой от более низкого компонента в виде отрисовки. Для этих целей может использоваться библиотека `RxJS`, предоставляющая семейство функций для оповещений и подписки на оповещения. Чаще всего эта библиотека используется в другом фреймворке для отрисовки - Angular. Но ввиду её большой популярности, простоты использования и применения подходов функционального программирования, её можно адаптировать и в React приложении при помощи специальных функций хуков, выносящих состояние из `RxJS` в состояние React. `RxJS` предоставляет большое количество утилитных вспомогательных функций мультикаста, сохранения изначального значения, подписки на состояния DOM-элементов, работу с HTTP и различных преобразований в функциональном стиле, что позволяет хранить бизнес логику чистой, ясной и понятной.

Для дополнительной обработки можно использовать библиотеку `ts-pattern`, которая позволяет проверить содержимое объекта по заданному паттерну.

Как уже было сказано ранее, API валидирует приходящие в неё данные. Часто хорошим тоном является перенос большинства правил на фронтенд, чтобы избежать отправки данных на сервер и не заставлять пользователя ждать. Существует множество библиотек, позволяющие задать парсинг для объектов. В данной ВКР можно использовать библиотеку `Yup` в связке с `react-hook-forms`, предоставляющую широкие возможности для задачи правил и сохранения состояния формы.

Процесс сборки обеспечивает библиотека `Vite` - относительно молодой сборщик, умеющий работать со многими фреймворками. Основным его отличием от устаревшего `Webpack` является гораздо большая скорость сборки. Но для `Vite` всё ещё достаточно мало вспомогательных модулей. Для данного проекта, в остальном, его будет достаточно.

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		25

На смену традиционному пакетному менеджер rpm приходит rpm. Это альтернативный пакетный менеджер, основная цель которого - альтернативный механизм построения дерева зависимостей, направленный на переиспользование ранее загруженных пакетов при помощи ссылок. rpm же обычно пытается загрузить все библиотеки, что влияет на его производительность а также на занимаемое место.

1.5.5 Инструмент для контейнеризации Docker

Docker - это набор программ, который используя средства виртуализации операционной системы, позволяет создать рабочее пространство для выполнения определённой программы внутри легковесных контейнеров.

Многие называют Docker полноценной виртуальной машиной, хотя это не совсем правда. Виртуальная машина создаёт с нуля всю операционную систему, запускает её ядро и затем транслирует команды поступающие из ядра в операционную систему. В итоге мы получаем изолированную систему, которая ничего не знает о родительской операционной системе. Docker же поступает немного иначе, он тоже инкапсулирует рабочее пространство, однако он это делает внутри процесса, который не создаёт отдельное ядро а напрямую общается с родительским ядром. Именно поэтому Docker контейнеры и сильно легковесней запуска полноценной виртуальной машины: это всё ещё нативные программы.

В репозитории Docker можно найти множество готовых образов для развёртывания. Контейнеризация происходит при помощи файла-рецепта (обычно он имеет название .Dockerfile), который описывает, как подготовить контейнер к работе. В него могут входить команды копирования, запуска Bash или Powershell скриптов, указания базовых контейнеров, на основе которых он и будет создавать новый контейнер. Сам контейнер подготавливается, каждый его шаг кешируется и в итоге мы получаем готовый к работе контейнер. Контейнер можно запустить, передав в него дополнительные настройки: подключенные дисковые пространства, проброс портов из дочернего контейнера в родительскую операционную систему, указание в пределах какой сети он может работать. Таким образом, Docker - инструмент, позволяющий инкапсулировать в себе рабочее пространство и обеспечить его кросс-выполняемость на разных машинах (разумеется, на которых можно установить Docker).

Более высокоуровневым компонентом, позволяющим собирать несколько Docker контейнеров и запускать их вместе, является Docker Compose. Файл конфигурации Docker Compose позволяет установить иерархию зависимостей между контейнерами: указать, какой контейнер зависит друг от друга. Например, бекенд

					ОБЗОР ПРЕДМЕТНОЙ ОБЛАСТИ	Лист
Изм.	Лист	№ докум.	Подп.	Дата		26

не может запустить без базы данных. А также указать для докер образов дисковые пространства, сети и прочее.

Список иллюстраций

**Список картинок нужен только для прохождения нормоконтроля.
В диплом он не вшивается!**

					Список иллюстраций			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.		Пахомов В.А.		3.06.26				
Руковод.		Осипов О.В.		3.06.26			1	27
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.		Бондаренко Т.В.		11.06.26				
Зав. каф.		Поляков В.М.		25.06.26				

Список таблиц

1.1	Сравнительная таблица сервисов хранения аудиоданных	12
1.2	Сравнительная таблица кодеков	15

Список таблиц нужен только для прохождения нормоконтроля.

В диплом он не вшивается!

					Список таблиц			
Изм.	Лист	№ докум.	Подп.	Дата				
Разраб.	Пахомов В.А.			3.06.26	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Руковод.	Осипов О.В.			3.06.26			2	27
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.	Бондаренко Т.В.			11.06.26				
Зав. каф.	Поляков В.М.			25.06.26				

Листинги

**Список листингов нужен только для прохождения нормоконтроля.
В диплом он не вшивается!**

					СОДЕРЖАНИЕ			
Изм.	Лист	№ докум.	Подп.	Дата	Разработка программного обеспечения для прогнозирования погоды на основе анализа спутниковых снимков с использованием методов машинного обучения	Лит.	Лист	Листов
Разраб.		Пахомов В.А.		3.06.26				
Руковод.		Осипов О.В.		3.06.26			27	27
Консул.						БГТУ им. В.Г. Шухова, ПБ-223		
Н. контр.		Бондаренко Т.В.		11.06.26				
Зав. каф.		Поляков В.М.		25.06.26				